WEEK 04 TECHNICAL REPORT

YCB-V Object Classification II

Intern	Mehrana Rezaei
Group	Group 1 — Basic ML Knowledge
Main Quest	Improving Deep Neural Networks
Side Quest	Object Classification II

1. Objective

The practical objective of Week 04 was to continue the Week 03 YCB-V object-classification baseline on the same five object classes and the same scene-level train/validation/test split, while testing whether regularization, training-only data augmentation, optimizer choice, learning-rate changes, and controlled hyperparameter tuning could improve generalization to unseen scenes. The model family was intentionally kept as an MLP so the effect of the Week 04 optimization choices could be compared directly with the Week 03 baseline.

All numerical values, tables, and model-output figures in this report were taken from the executed week04_object_classification_improvement notebook supplied for this report; no Week 04 metric was filled with an invented value.

2. Main Quest Progress and Relevant Concepts

The Main Quest for this week was Improving Deep Neural Networks. The course-progress screenshots supplied for the report show study of the optimization-algorithm block and the hyperparameter-tuning / batch-normalization / multiclass-classification block of Andrew Ng's course material.

2.1 Optimization lessons studied

- 21 — Mini-batch Gradient Descent
- 22 — Understanding Mini-batch Gradient Descent
- 23 — Exponentially Weighted Averages
- 24 — Understanding Exponentially Weighted Averages
- 25 — Bias Correction in Exponentially Weighted Averages
- 26 — Gradient Descent with Momentum
- 27 — RMSprop
- 28 — Adam Optimization Algorithm
- 29 — Learning Rate Decay
- 30 — The Problem of Local Optima
- 31 — Interview with Yuanqing Lin

2.2 Hyperparameter tuning, normalization, and multiclass lessons studied

- 35 — Tuning Process
- 36 — Using an Appropriate Scale to Pick Hyperparameters
- 37 — Hyperparameter Tuning in Practice: Pandas vs. Caviar
- 38 — Normalizing Activations in a Network
- 39 — Fitting Batch Normalization into a Neural Network
- 40 — Why Does Batch Normalization Work?
- 41 — Batch Normalization at Test Time

- 42 — Softmax Regression
- 43 — Training a Softmax Classifier
- Deep Learning Frameworks (next lesson shown in the supplied course screenshot)

2.3 Concepts applied to the Side Quest

The optimizer and hyperparameter-tuning lessons were directly reflected in the Week 04 experiment design. The notebook compared Adam, AdamW, and SGD with momentum, tested two learning-rate settings, and kept the experiments controlled by holding the dataset split, batch size, architecture width, loss function, and early-stopping rule constant. Mini-batch training was used with batch size 64. Dropout and weight decay were added as regularization, and augmentation was introduced only for the training split in selected experiments.

The Batch Normalization and Softmax lessons were relevant conceptual preparation for later neural-network work. Batch normalization was studied but was not added as another changing factor in this notebook, because the Week 04 experiment set focused on regularization, augmentation, optimizer, and learning-rate changes. The five-class output remains a multiclass classification problem, making the Softmax-classification material directly relevant to interpreting the model's class scores and training objective.

3. Week 03 Baseline and Motivation

The measured Week 03 baseline had best validation loss 0.0516, best validation accuracy 0.9813, and test accuracy 0.5133. Although validation accuracy was very high, test accuracy was much lower. This large validation-to-test gap was the main reason Week 04 focused on generalization rather than simply increasing training accuracy.

Metric	Week 03	Week 04 selected model	Change
Best validation loss	0.0516	0.0327	−0.0189 (lower/better)
Best validation accuracy	0.9813	0.9893	+0.0080
Final test accuracy	0.5133	0.4178	−0.0955

4. Method

The notebook reused the same lightweight YCB-V BOP'19 subset and rebuilt the same object-instance table used in Week 03. Candidate object instances were retained only when a bounding box was available, the box width and height were at least 24 pixels, and the visible fraction was at least 0.35. Five classes were selected from classes represented in at least three scenes. Object crops used 8% padding around the ground-truth bounding box and were resized to 64×64 RGB before normalization.

The split remained scene-level: all samples from a scene were placed in only one of train, validation, or test. The notebook reproduced the Week 03 split exactly: 877 training samples from 8 scenes, 375 validation samples from 2 scenes, and 450 test samples from 2 scenes. Hyperparameter selection used validation loss; the final test set was evaluated only after the best experiment had been selected.

4.1 Selected classes and available instances

Class	Object ID	Usable samples	Scenes	Mean visibility
tomato_soup_can	4	427	6	0.9574
sugar_box	3	375	5	0.8893
master_chef_can	1	300	4	0.8995
tuna_fish_can	6	300	4	0.9985
bleach_cleanser	12	300	4	0.8758

The executed notebook found 4,099 usable object instances before selecting the five classes above.

4.2 Reproduced scene-level split

Class	Train	Validation	Test
master_chef_can	150	75	75
sugar_box	225	75	75
tomato_soup_can	202	75	150
tuna_fish_can	150	75	75
bleach_cleanser	150	75	75
Total	877	375	450

Scene counts: train = 8 scenes, validation = 2 scenes, test = 2 scenes. The notebook explicitly verified that the expected Week 03 split size was reproduced.

5. Implementation

The classifier kept the Week 03 MLP family: a flattened 64×64×3 RGB crop was passed through fully connected layers of 256 and 128 hidden units with ReLU activations, followed by a five-class output layer. Week 04 parameterized dropout so the regularized experiments could place dropout after each hidden activation. Cross-entropy loss was used for all runs. Maximum training length was 20 epochs with early-stopping patience of 5 epochs.

Pipeline stage	Implementation	Purpose
Object crop	Ground-truth box + 8% padding; resize to 64×64	Keep the same Week 03 input representation
Normalization	Per-channel mean/std = 0.5 / 0.5	Standardize model input range
Augmentation	Horizontal flip, affine ±10°, translation/scale, color jitter	Training-only robustness experiment
Architecture	Flatten → 256 ReLU → dropout → 128 ReLU → dropout → 5 outputs	Controlled MLP comparison with Week 03
Training	Batch size 64; max 20 epochs; early stopping patience 5	Comparable optimization procedure
Selection	Lowest validation loss	Avoid selecting on test performance

5.1 Controlled experiments

ID	Main change	Optimizer	LR	Dropout	Other settings
E1	Regularization only	Adam	1e−3	0.30	WD=1e−4; Aug=No
E2	Regularization + augmentation	Adam	1e−3	0.30	WD=1e−4; Aug=Yes
E3	Lower learning rate	Adam	3e−4	0.30	WD=1e−4; Aug=Yes
E4	AdamW optimizer	AdamW	3e−4	0.30	WD=1e−4; Aug=Yes
E5	SGD + momentum	SGD	1e−2	0.30	WD=1e−4; Aug=Yes; momentum=0.9

6. Results

E1_regularization was selected because it achieved the lowest validation loss. Its selected checkpoint was epoch 7 with validation loss 0.0327, validation accuracy 0.9893, and training accuracy 1.0000. The train–validation accuracy gap at that checkpoint was only 0.0107, but the final unseen-scene test accuracy was 0.4178. This mismatch is a central finding of Week 04.

6.1 Controlled-experiment comparison

ID	Best epoch	Best val loss	Best val acc	Train acc	Train−val gap	Epochs run
E1	7	0.032718	0.9893	1.0000	0.0107	12
E2	4	0.063640	0.9787	0.9886	0.0099	9

ID	Best epoch	Best val loss	Best val acc	Train acc	Train−val gap	Epochs run
E3	10	0.081510	0.9680	0.9977	0.0297	15
E4	2	0.097444	0.9760	0.9624	−0.0136	7
E5	3	0.129741	0.9413	0.9510	0.0096	8

6.2 Final test metrics

Metric	Observed value
Final test accuracy	0.4178
Macro F1	0.42 (classification-report rounding)
Weighted F1	0.43
Week 03 test accuracy	0.5133
Change vs Week 03	−0.0955

6.3 Per-class performance

Class	Precision	Recall	F1-score	Support
tomato_soup_can	0.41	0.49	0.45	150
sugar_box	0.71	0.39	0.50	75
master_chef_can	1.00	0.91	0.95	75
tuna_fish_can	0.11	0.08	0.09	75
bleach_cleanser	0.11	0.15	0.12	75
Macro average	0.47	0.40	0.42	450
Weighted average	0.46	0.42	0.43	450

6.4 Representative Visualizations

The figures below are outputs embedded in the executed notebook. They show the training-only augmentation policy, training behavior across all controlled experiments, validation comparison, final confusion matrix, and representative correct/incorrect predictions.



Figure 1. Training-only augmentation preview for a representative object crop.

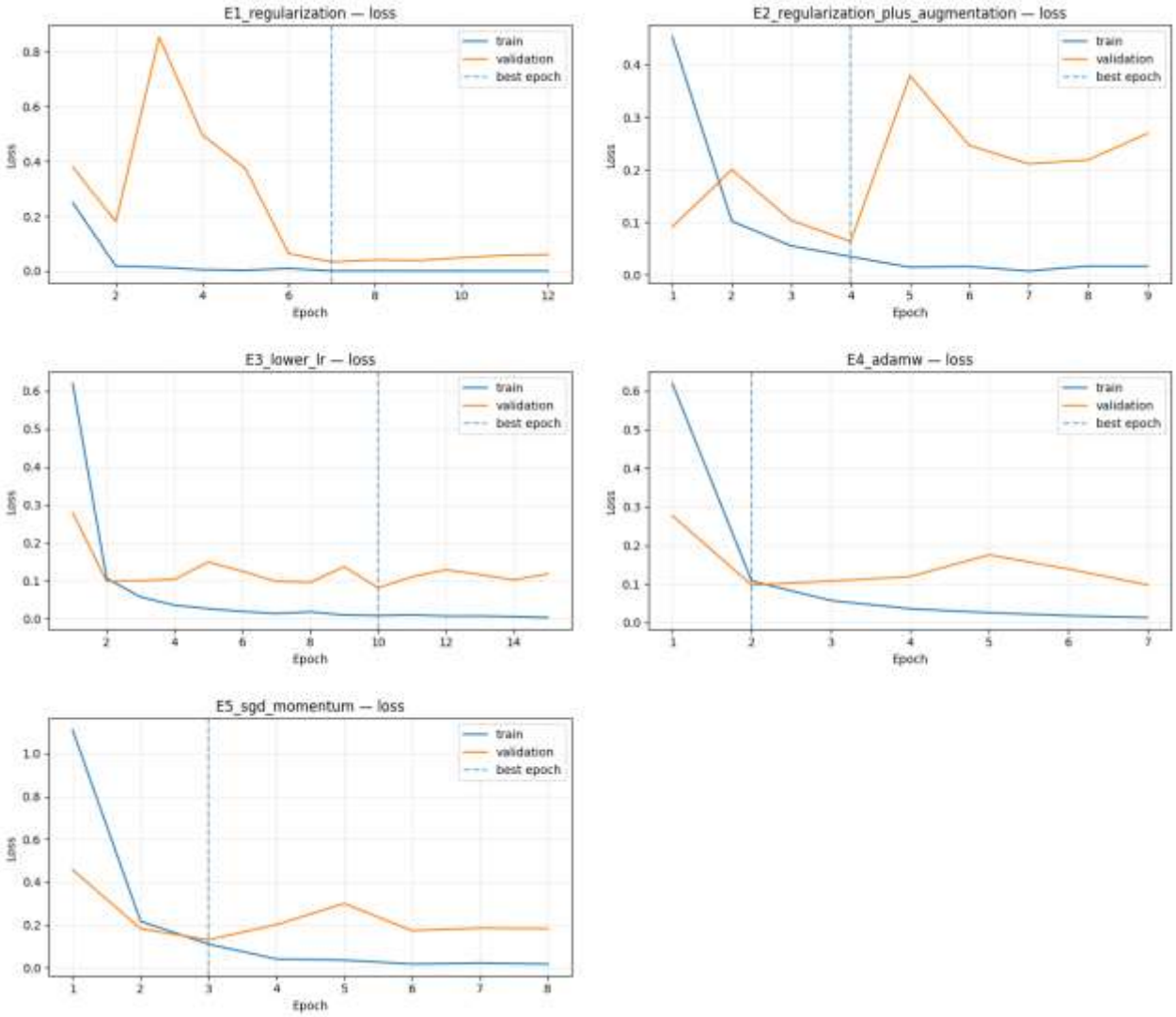


Figure 2. Training and validation loss curves for E1–E5. Dashed vertical lines mark each experiment's selected epoch.

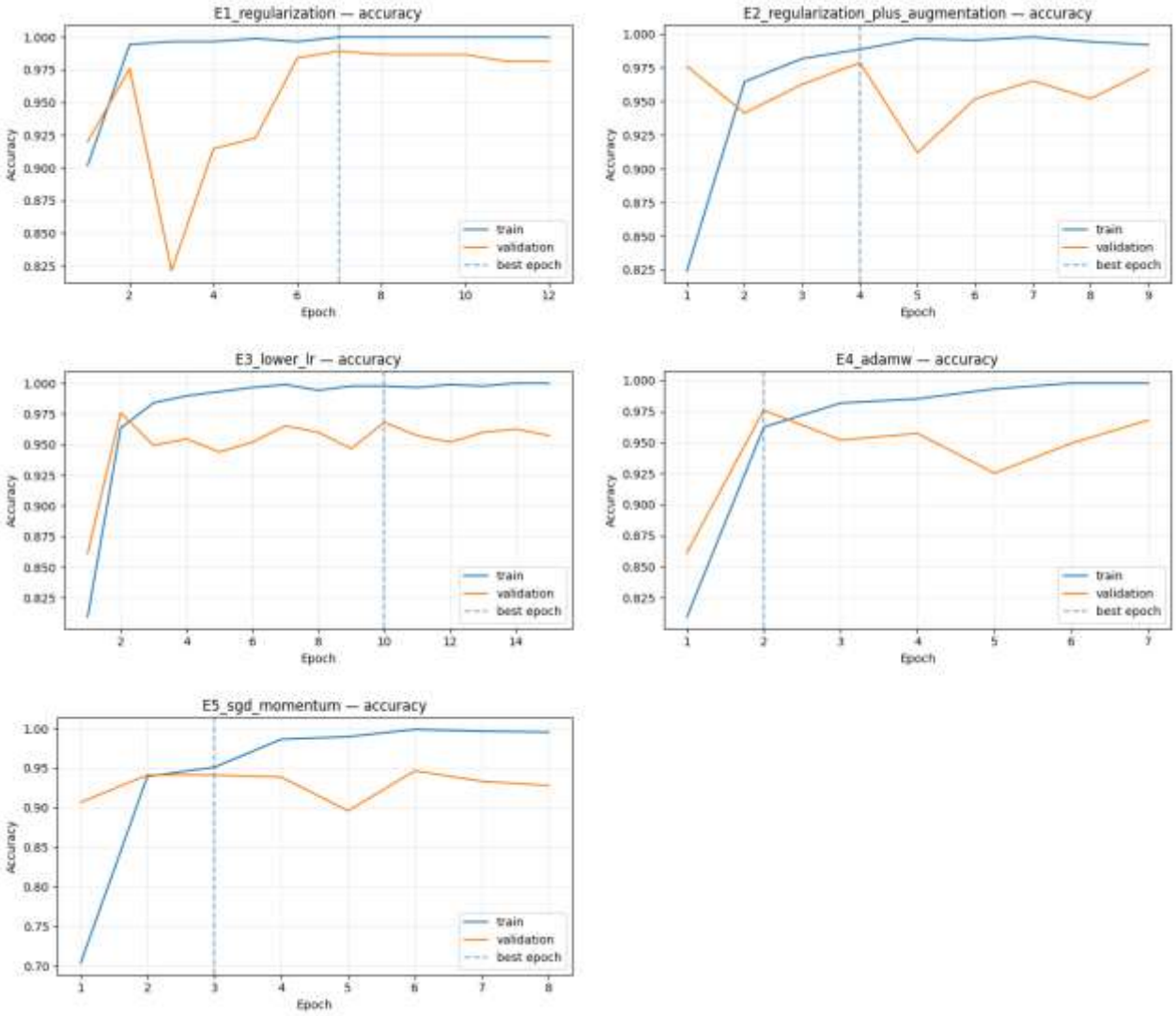


Figure 3. Training and validation accuracy curves for E1–E5.

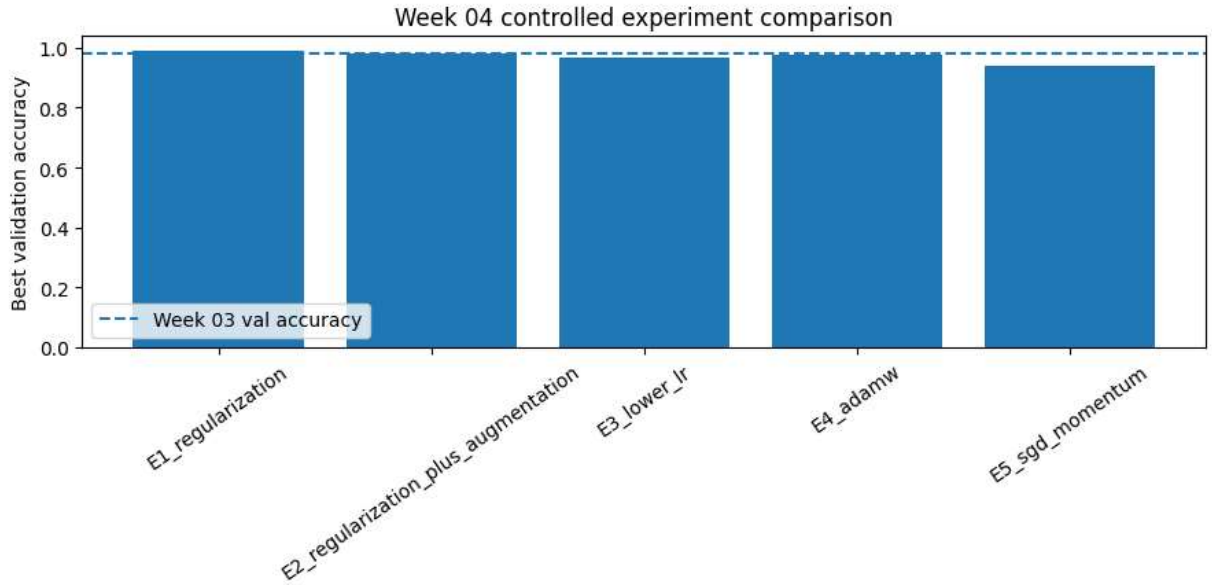


Figure 4. Best validation accuracy across the five controlled Week 04 experiments; the dashed line is the Week 03 validation-accuracy reference.

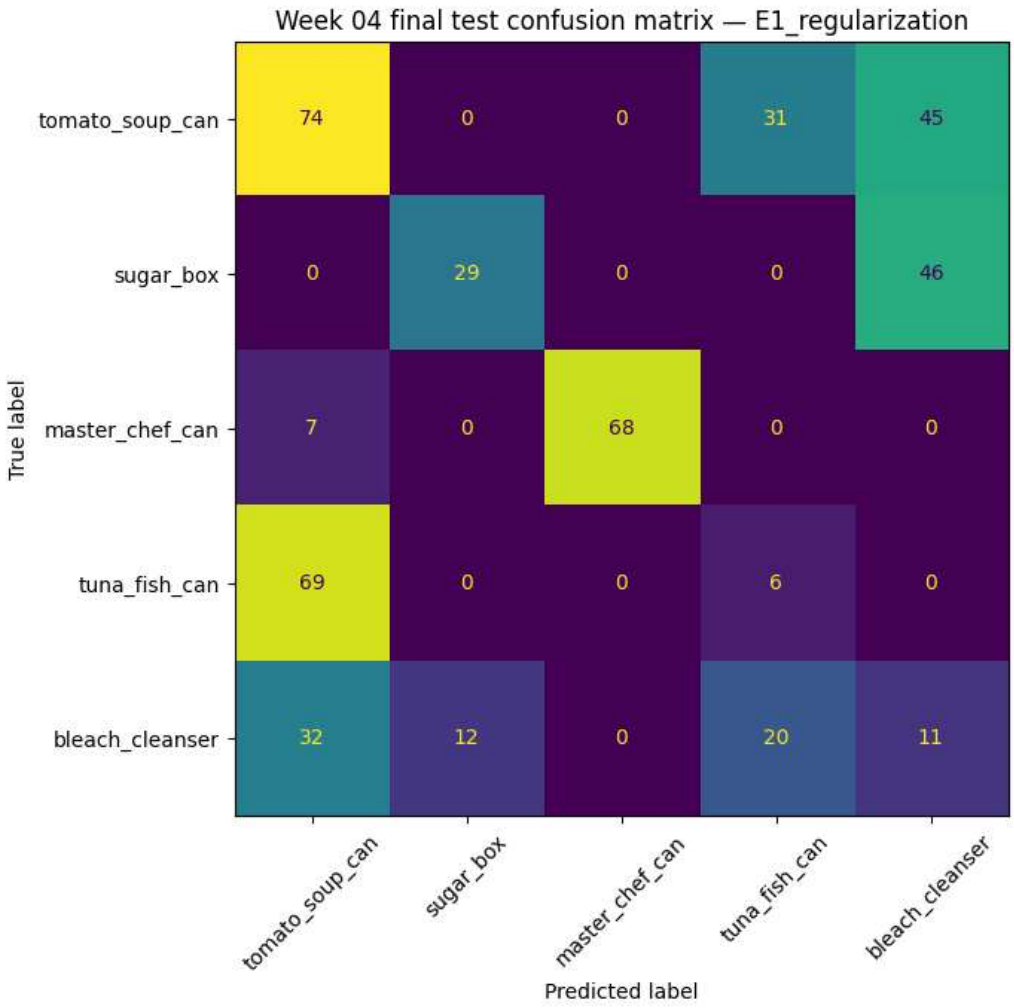


Figure 5. Confusion matrix on the final held-out test set for the selected E1_regularization checkpoint.

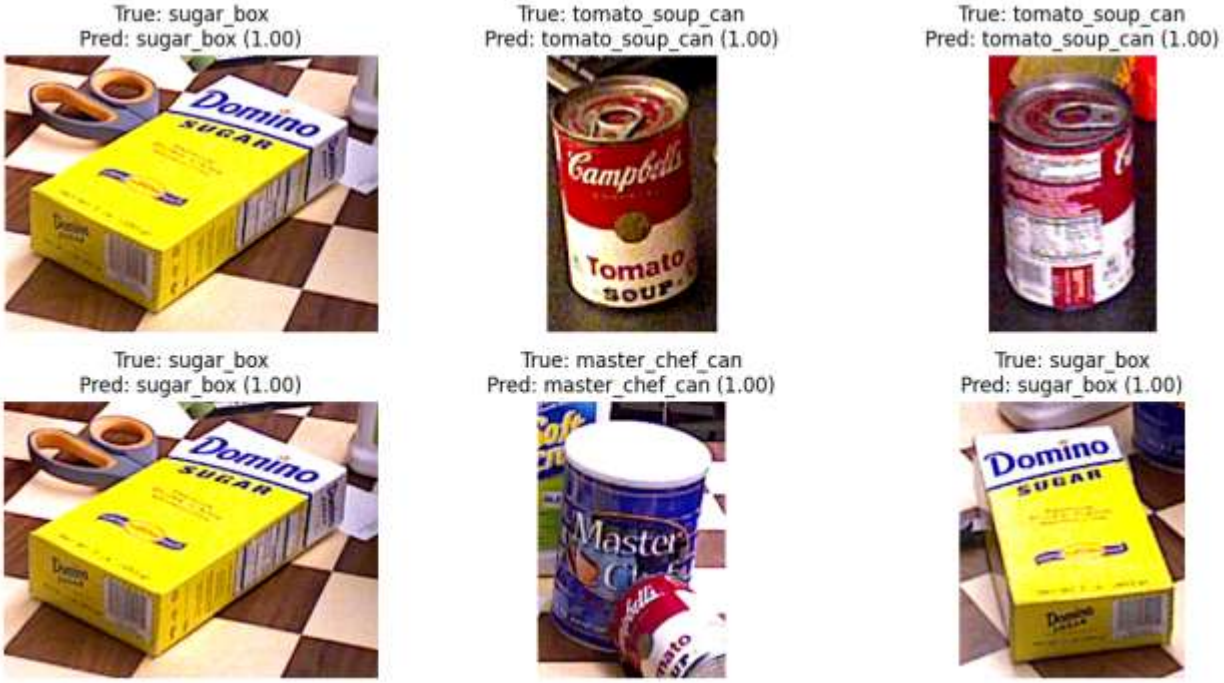


Figure 6. Representative correct test predictions saved by the notebook.



Figure 7. Representative incorrect test predictions saved by the notebook. Several failures are high-confidence mistakes.

6.5 Observations from the outputs

The training curves show that the MLP can fit the training data extremely well: E1 reached training accuracy 1.0000 at the selected checkpoint. Validation accuracy was also very high for E1 (0.9893), and all five experiments reached reasonably high validation accuracy. However, that validation performance did not transfer to the held-out test scenes.

The confusion matrix shows that master_chef_can was substantially easier than the other classes: 68 of 75 samples were classified correctly. In contrast, tuna_fish_can had only 6 correct predictions out of 75 and was predicted as tomato_soup_can 69 times. bleach_cleanser had 11 correct predictions, while 32 were predicted as tomato_soup_can, 12 as sugar_box, and 20 as tuna_fish_can. sugar_box also showed a strong failure mode toward bleach_cleanser (46 of 75 samples).

The representative failure images provide qualitative evidence that scene context and crop contamination deserve closer inspection. For example, some bleach-cleanser crops visibly contain part of the master-chef can, and several wrong predictions are made with confidence near 1.00. This does not prove a single failure cause, but it supports the hypothesis that the raw-pixel MLP can learn scene/viewpoint/background shortcuts that do not generalize reliably to new scenes.

7. Failure Analysis and Interpretation

Week 04 improved the validation objective but did not improve the actual held-out test result. Relative to Week 03, best validation loss improved from 0.0516 to 0.0327 and validation accuracy improved from 0.9813 to 0.9893, yet test accuracy decreased from 0.5133 to 0.4178. The validation-to-test accuracy gap for the selected Week 04 checkpoint was therefore about 0.5715, even though its train-to-validation gap was only 0.0107.

This result means the small train–validation gap alone was not a sufficient signal of generalization. The two validation scenes appear easier or more similar to the training scenes than the two final test scenes. Because the model selection rule used validation loss only, E1 was correctly selected according to the experiment protocol, but the selected validation metric was not sufficiently representative of unseen-scene performance.

7.1 What did not work

- Adding dropout = 0.30 and weight decay = 1e−4 improved validation loss but did not improve final test accuracy.

- Training-only augmentation did not beat E1 on validation loss; E2 reached validation loss 0.0636 and accuracy 0.9787.
- Lowering Adam learning rate to 3e−4 (E3) reduced convergence speed and did not improve validation performance.
- AdamW (E4) and SGD with momentum (E5) did not outperform Adam E1 under the same split and architecture family.
- The raw-pixel MLP still reached near-perfect training fit, indicating that optimization capacity was not the limiting factor.
- High-confidence test errors and strong class confusions indicate a generalization/data-mismatch problem that optimizer tuning alone did not solve.

7.2 Questions carried into Week 05

- Which classes and scene conditions account for most of the test error?
- Are crops containing other objects or large amounts of background causing shortcut learning?
- How much of the error is associated with viewpoint, scale, clutter, occlusion, and visibility changes?
- Are the validation scenes representative enough for model selection, or is there a scene/domain mismatch?
- Are there suspicious labels or bounding-box/crop cases that should be inspected manually before changing the model again?

8. Conclusion and Next Step

Week 04 completed the required controlled improvement study on the Week 03 classifier. Regularization, augmentation, learning-rate changes, AdamW, and SGD with momentum were tested while the data split and MLP family were held fixed. E1_regularization produced the best validation loss (0.0327) and validation accuracy 0.9893, but its final test accuracy was 0.4178, below the Week 03 baseline test accuracy of 0.5133.

The main lesson is that better validation metrics did not guarantee better performance on unseen YCB-V scenes. The evidence points toward scene/domain mismatch and class-specific failure modes rather than a simple optimizer problem. For Week 05, the correct next step is structured error analysis: quantify class-wise mistakes, inspect difficult and high-confidence failures, check crop/background contamination and possible label issues, and rank the most important sources of error before making additional model changes.